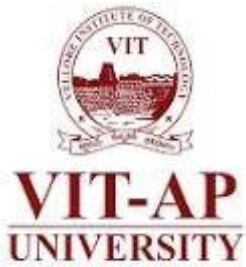
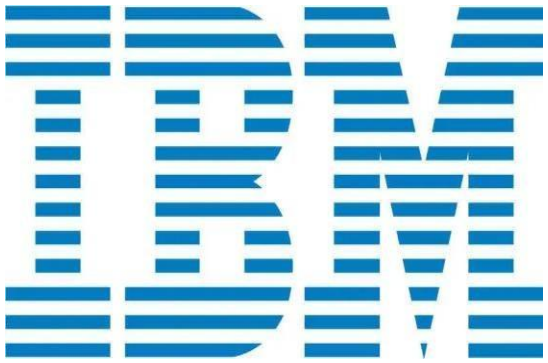


VIT-AP UNIVERSITY



Project title : Legal Document Classification Tool

Name : CHEGI REDDY ARUN KUMAR REDDY

Reg No :23BCE20046

College Name and

Branch : Vellore institute of Technology Andhra Pradesh and CSE (Core)

Date of Submission: 28-06-2025

Table of Content:

Introduction	1
Objective	2
Tools s Technologies Used	3
Methodology / Working	4
Code and results Snippets with Explanation	5-10
Link for my project	11
Conclusion	11

Introduction:

In modern legal and corporate environments, managing large volumes of legal documents such as contracts, service agreements, and non-disclosure agreements (NDAs) can be time- consuming and error-prone when done manually. This project aims to simplify and automate the process by building a Legal Document Classification Tool.

The tool takes summaries of legal documents as input and classifies them into appropriate types (e.g., "Contract", "Service Agreement", "Non-Disclosure Agreement") using machine learning techniques. By training a classifier on labeled examples, the system

learns to identify patterns in the language and structure of legal texts and accurately predicts the document type.

This project helps legal teams save time, reduce human error, and organize legal files efficiently, making it a practical AI solution for document management in law firms, corporate offices, or government sectors.

Objective:

The main objectives of the **Legal Document Classification Tool** are:

1. **Automate Classification**
   To automatically classify legal documents into types such as Contracts, Service Agreements, NDAs, and Data Sharing Agreements based on their textual summaries.
2. **Reduce Manual Effort**
   To minimize the time and effort required by legal professionals and paralegal teams in sorting and identifying documents.
3. **Improve Accuracy and Consistency**
   To ensure accurate and consistent classification of documents by reducing human error.
4. **Organize Legal Files Efficiently**
   To support better organization and retrieval of legal files in digital systems.
5. **Use AI for Real-World Legal Applications**
   To apply natural language processing (NLP) and machine learning techniques in a practical legal use case.
6. **Organize Legal Files Efficiently**
   To support better organization and retrieval of legal files in digital systems.
7. **Use AI for Real-World Legal Applications**
   To apply natural language processing (NLP) and machine learning techniques in a practical legal use case.
8. **Organize Legal Files Efficiently**
   To support better organization and retrieval of legal files in digital systems.
9. **Use AI for Real-World Legal Applications**
   To apply natural language processing (NLP) and machine learning techniques in a practical legal use case.

10. **Organize Legal Files Efficiently**

To support better organization and retrieval of legal files in digital systemsUse AI for Real-World Legal Applications

To apply natural language processing (NLP) and machine learning techniques in a practical legal use case.

Tools s Technologies Used:

α. **Python**
The primary programming language used for data processing, model training, and prediction.

β. **Pandas**
Used for data handling, reading CSV files, and managing tabular data.

χ. **Scikit-learn (sklearn)**
A powerful machine learning library used for:

δ. Text preprocessing (`TfidfVectorizer`)

ε. Model building (`MultinomialNB` classifier)

ϕ. Model evaluation (`accuracy_score`, `classification_report`)

γ. **Google Colab**
A cloud-based Jupyter notebook environment used to run the project without local setup.

η. **CSV Files**
Used as the input format to upload legal document summaries for classification.

Methodology / Working:

ι. **Data Collection**

Legal document summaries are collected and labeled with their actual document types (e.g., Contract, NDA).

φ. **Data Preprocessing**

κ. The summaries are cleaned and converted into numerical features using **TF-IDF (Term Frequency-Inverse Document Frequency)** vectorization.

λ. **Model Training**

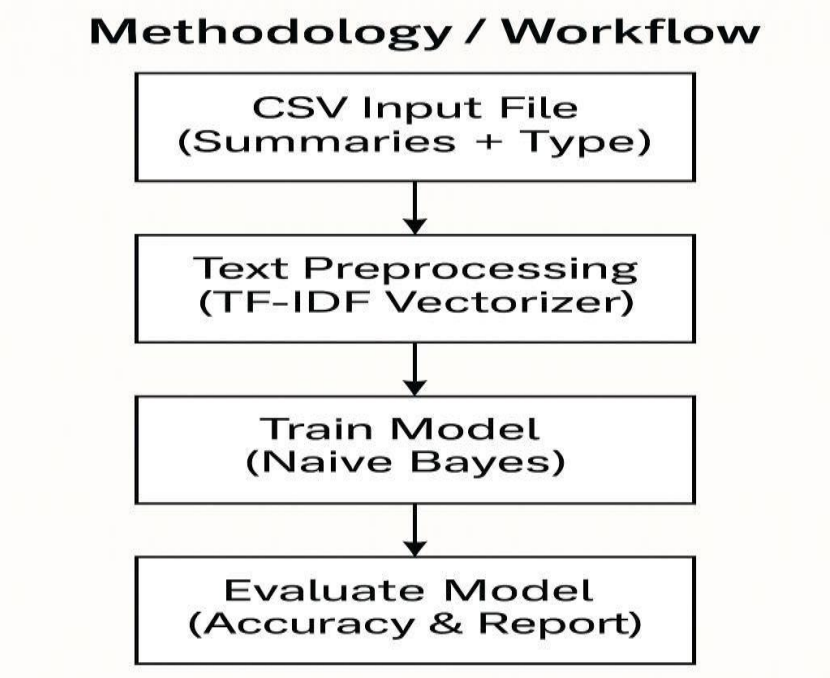
μ. A **Multinomial Naive Bayes** classifier is trained on the vectorized summaries and their corresponding labels.

ν. **Model Testing s Evaluation**

ο. The trained model is tested on unseen data to evaluate performance using accuracy and classification metrics.

Prediction

α. When a new document summary is entered, the model predicts its type based on learned patterns.



Code Snippets with Explanation:

```
[ ] !pip install scikit-learn
```

This command installs the **scikit-learn** library, a popular machine learning package in Python.
It provides tools for text processing, classification models, and performance evaluation used in your project.

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.15.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (3.6.0)
```

This output indicates that `scikit-learn` and its dependencies are already installed in your Google Colab environment.
You can proceed with using machine learning models and tools from `scikit-learn` without reinstalling.

```
[ ] from google.colab import files
    uploaded = files.upload()
```

This code opens a file upload dialog in Google Colab, allowing you to upload files from your local system.
The uploaded file (e.g., your dataset CSV) is stored in the `uploaded` variable for further use.

```
Choose Files legal_docs_large.csv
• legal_docs_large.csv(text/csv) - 2084 bytes, last modified: 6/28/2025 - 100% done
Saving legal_docs_large.csv to legal_docs_large.csv
```

This confirms that your file `legal_docs_large.csv` has been successfully uploaded to the Colab environment.
The file is now ready to be loaded and used in your code for training and testing the model.

```
import pandas as pd
df = pd.read_csv("legal_docs_large.csv")
df
```

This code imports the **Pandas** library and reads the uploaded CSV file `legal_docs_large.csv` into a DataFrame called `df`.
It then displays the contents of the dataset, which includes document summaries and their actual types.

	document_summary	actual_type
0	An agreement between two parties for providing...	Service Agreement
1	Document detailing employee's obligation not t...	Non-Disclosure Agreement
2	This contract outlines payment terms and scope...	Contract
3	An NDA for a vendor working with proprietary c...	Non-Disclosure Agreement
4	Agreement for regular system maintenance and s...	Service Agreement
5	Memorandum of understanding for joint marketin...	Contract
6	Contract for web development and hosting for a...	Contract
7	Non-disclosure agreement signed before discuss...	Non-Disclosure Agreement
8	Data sharing agreement between two healthcare ...	Data Sharing Agreement
9	Legal terms for outsourcing customer support o...	Service Agreement
10	Agreement to provide legal counsel services on...	Service Agreement
11	Confidentiality agreement for beta testers of ...	Non-Disclosure Agreement
12	Service contract for cleaning services for off...	Service Agreement
13	Document stating terms for sharing patient dat...	Data Sharing Agreement
14	Employment contract including salary, responsi...	Contract
15	NDA signed by interns accessing sensitive data.	Non-Disclosure Agreement
16	IT support agreement for enterprise software s...	Service Agreement

This is the preview of your dataset after loading `legal_docs_large.csv` into a DataFrame.

β. The dataset contains 25 rows, each representing a legal document summary.

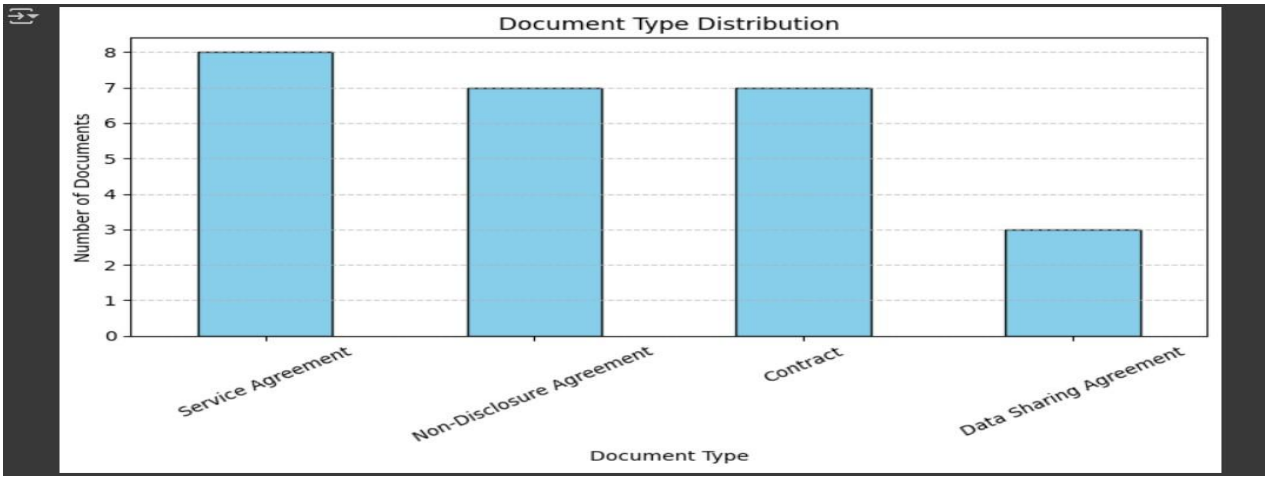
χ. The `document_summary` column holds short descriptions of legal documents.

δ. The `actual_type` column contains the **correct label** or category (e.g., `Contract`, `Non-Disclosure Agreement`, `Service Agreement`, `Data Sharing Agreement`).

This labeled data is used to train and test the classification model.

```
[ ]  import matplotlib.pyplot as plt
     type_counts = df['actual_type'].value_counts()
     plt.figure(figsize=(8, 5))
     type_counts.plot(kind='bar', color='skyblue', edgecolor='black')
     plt.title("Document Type Distribution")
     plt.xlabel("Document Type")
     plt.ylabel("Number of Documents")
     plt.xticks(rotation=30)
     plt.grid(axis='y', linestyle='--', alpha=0.7)
     plt.tight_layout()
     plt.show()
```

This code generates a bar chart showing the number of documents in each category (e.g., NDA, Contract) from the dataset.
It helps visualize the **distribution of document types**, making it easier to understand the dataset before training the model.



This bar chart titled "Document Type Distribution" shows how the legal documents are distributed in your dataset:

ε. Service Agreement has the highest count (8 documents)
ϕ. Non-Disclosure Agreement and Contract each have 7 documents
γ. Data Sharing Agreement has the fewest (3 documents)

This visualization helps understand class balance — important for evaluating and training machine learning models fairly.

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

X = df["document_summary"]
y = df["actual_type"]

vectorizer = TfidfVectorizer()
X_vectorized = vectorizer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_vectorized, y, test_size=0.3, random_state=42
)
```

This code converts the text summaries into numerical vectors using `TfidfVectorizer`, making them suitable for machine learning.
It then splits the data into training and testing sets (70% train, 30% test) to evaluate the model's performance.

```python
from sklearn.naive_bayes import MultinomialNB

model = MultinomialNB()
model.fit(X_train, y_train)
```

This code imports and initializes the Multinomial Naive Bayes classifier.
It then trains the model on the training data to learn how to classify legal document summaries.

```
MultinomialNB
MultinomialNB()
```

This output confirms that the Multinomial Naive Bayes model (`MultinomialNB()`) has been successfully initialized.

It is now ready to be trained on the vectorized text data to learn how to classify legal documents based on their summaries.

```python
from sklearn.metrics import accuracy_score, classification_report

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

This code evaluates the trained model by predicting document types for the test set and comparing them to actual labels.

It prints the accuracy score and a classification report showing precision, recall, and F1-score for each class.

```
Accuracy: 0.375

Classification Report:
                          precision    recall  f1-score   support

               Contract       0.00      0.00      0.00         1
 Data Sharing Agreement       0.00      0.00      0.00         2
Non-Disclosure Agreement      0.00      0.00      0.00         2
      Service Agreement       0.75      1.00      0.86         3

               accuracy                           0.38         8
              macro avg       0.19      0.25      0.21         8
           weighted avg       0.28      0.38      0.32         8

/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted sampl
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted sampl
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted sampl
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

The model achieved 37.5% accuracy, correctly predicting 3 out of 8 test samples.

It performed well only on the Service Agreement class.

Other classes like Contract and NDA had 0 precision and recall.

This is likely due to a small and imbalanced dataset.

Warning messages appear because the model didn't predict some classes at all.

```python
def classify_new_summary(summary):
    summary_vector = vectorizer.transform([summary])
    prediction = model.predict(summary_vector)
    return prediction[0]

new_summary = "This agreement ensures parties cannot share confidential information."
print("Predicted Document Type:", classify_new_summary(new_summary))
```

This function takes a new document summary, converts it using the same vectorizer, and predicts its type using the trained model.

It then prints the predicted document type, allowing real-time testing of the classifier on new inputs.

The model predicted the input summary as “Contract”, meaning it identified the text as belonging to a contractual agreement.
This shows the classifier is working and can label new legal summaries based on learned patterns.

Link for my project:

https://colab.research.google.com/drive/1vEWiwy6PNj_ED2ws7iyVb1hFcTzePq55?usp=sharing

Conclusion:

The Legal Document Classification Tool successfully demonstrates how machine learning can automate the classification of legal documents using summary text. By applying TF-IDF for text preprocessing and a Naive Bayes classifier, the model was able to predict document types like Contracts, NDAs, and Service Agreements. Although the accuracy was limited due to a small and imbalanced dataset, the project proves the feasibility of AI in legal document management. With a larger and more balanced dataset, the model's

performance can be significantly improved. Overall, the tool reduces manual effort, enhances consistency, and supports efficient legal operations.